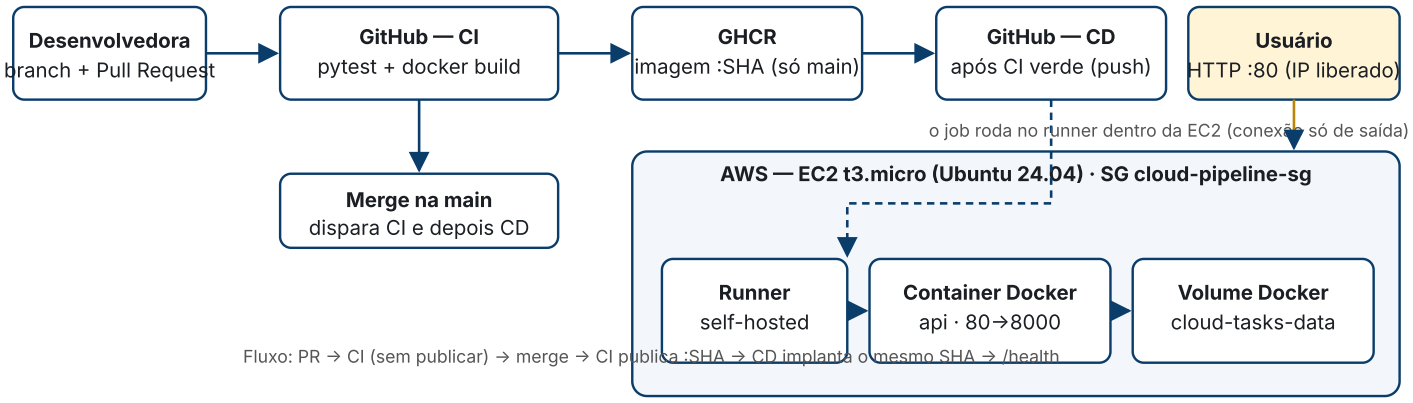
Relatório Final — Pipeline CI/CD da Cloud Tasks API na AWS

Disciplina: Computação em Nuvem | **Aluna:** Leticia Seto (trabalho individual) | **Repositório:** github.com/leticiaseto/cloud-tasks-api | **Data:** 13/11/2026

1. Visão geral e arquitetura

O projeto automatiza o caminho entre um commit e a aplicação em execução: uma API Flask/SQLite de tarefas é testada, empacotada em imagem Docker, publicada no GitHub Container Registry (GHCR) e implantada em uma instância EC2. A imagem de cada deploy é identificada pelo SHA do commit que passou nos testes, o que torna cada versão publicada rastreável e imutável.



2. Infraestrutura, portas e recursos

Recurso	Configuração
Instância EC2	`cloud-pipeline-api`, tipo t3.micro, Ubuntu Server 24.04 LTS (AMI noble-24.04 amd64), região us-east-1, IPv4 público atribuído automaticamente, par de chaves próprio.
Software	Docker Engine 29.9.3 (repositório oficial) com o usuário `ubuntu` no grupo `docker`; runner do GitHub Actions v2.338.2 instalado como serviço.
Grupo de segurança `cloud-pipeline-sg`	**Entrada:** TCP 22 (SSH) e TCP 80 (HTTP), ambas restritas a endereços `/32` do administrador — nunca `0.0.0.0/0`. **Saída:** todo o tráfego (necessário para o runner falar com o GitHub e para baixar imagens).
Portas	Contêiner escuta em 8000 (gunicorn); a porta 80 do host é mapeada para 8000. O runner usa apenas HTTPS de saída (443); nenhuma porta de entrada é aberta para o GitHub.
Persistência	Volume Docker `cloud-tasks-data` montado em `/data`; a aplicação grava o SQLite em `DB_PATH=/data/tasks.db`.
Imagem	`ghcr.io/leticiaseto/cloud-tasks-api:<SHA do commit>`; pacote público para permitir `docker pull` anônimo no deploy.

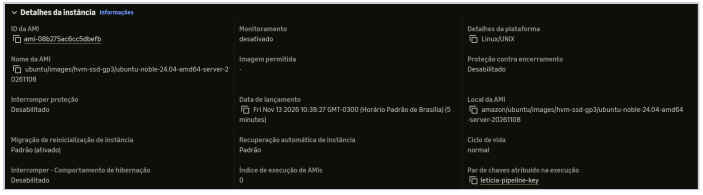


Fig. 1 — Detalhes da instância: AMI Ubuntu 24.04, plataforma Linux, par de chaves (ID da conta omitido).

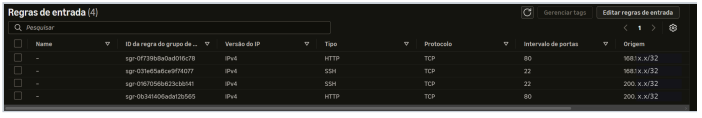


Fig. 2 — Regras de entrada: SSH (22) e HTTP (80) só para IPs /32 (octetos finais ocultados).

3. Pipeline de CI/CD

CI (`ci.yml`) roda em *pull request* e em *push* na `main`. O job `test` executa `pytest` (9 testes); o job `image` faz o `docker build` sempre, mas só autentica no GHCR e publica a imagem quando o evento é *push* na `main`, com a tag igual ao SHA do commit. Assim, um PR nunca publica nem implanta nada.

CD (`cd.yml`) é disparado por `workflow_run` ao término do CI e só executa se o CI foi bem-sucedido e originado por *push*. O job usa o ambiente `production`, faz o checkout do mesmo `head_sha` do CI, executa `scripts/deploy.sh` com a imagem `ghcr.io/leticiaseto/cloud-tasks-api:<SHA>` (cria o volume, baixa a imagem, recria o contêiner e aguarda o `/health`) e, por fim, confere `/health` com `curl`. Um grupo de concorrência impede dois deploys simultâneos.

Decisão — runner self-hosted em vez de SSH. O modelo original do CD conectava por SSH a partir de um runner hospedado pelo GitHub, o que exigiria liberar a porta 22 para as faixas de IP do GitHub (amplas e variáveis) ou guardar uma chave privada nos segredos. Optei por instalar um **runner self-hosted na própria EC2**: ele só abre conexões de saída, então a porta 22 permanece restrita ao meu IP e nenhuma chave privada ou endereço da instância precisa ser armazenado no GitHub. *Riscos assumidos:* o runner executa código do repositório dentro da instância e o grupo `docker` equivale a privilégios elevados. Mitigações: repositório com aprovação obrigatória para workflows de colaboradores externos, CD condicionado a CI verde de *push* na `main` e instância descartável, encerrada ao final.

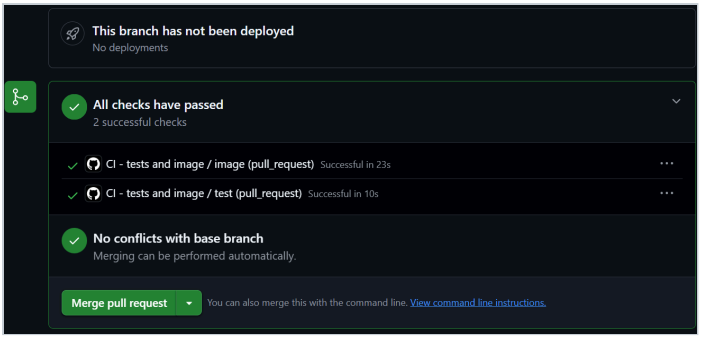


Fig. 3 — PR #1: checks aprovados e "No deployments" — o PR testa e constrói, mas não publica nem implanta.

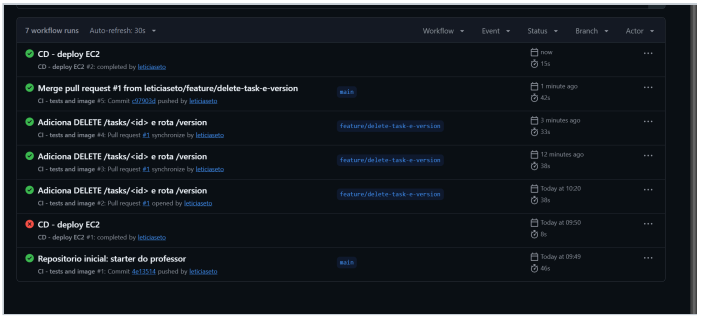


Fig. 4 — Execuções: CD #1 vermelho (esperado, ainda com o modelo SSH sem segredos); CI dos PRs verdes; CI #5 no merge e CD #2 verde.

4. Segurança e segredos

- Nenhum segredo de acesso foi cadastrado: o CD usa o `GITHUB_TOKEN` apenas no CI (`packages: write` restrito ao job `image`) e o deploy ocorre localmente na EC2.
- Chave `.pem` e capturas de tela mantidas fora do repositório (`.gitignore`); o repositório exige aprovação para workflows de colaboradores externos (Fig. 7).
- SSH e HTTP limitados a IPs /32; pacote GHCR público porque a imagem não contém dados nem credenciais.

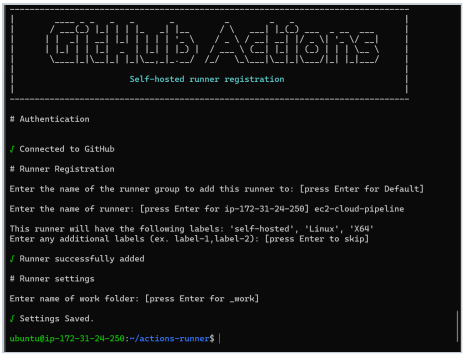


Fig. 5 — Registro do runner na EC2 (`Settings Saved`); token omitido.

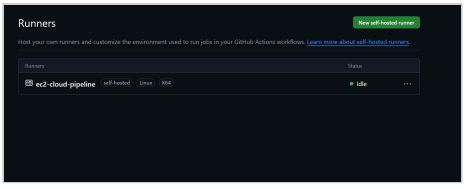


Fig. 6 — Runner `ec2-cloud-pipeline` em estado Idle.

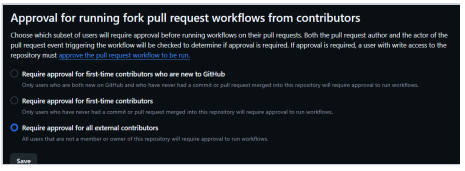


Fig. 7 — Aprovação exigida para todos os colaboradores externos.

5. Alterações na aplicação

- `DELETE /tasks/<id>`: devolve **204** quando remove e **404** (JSON de erro) quando a tarefa não existe; 3 testes (remove só a tarefa pedida, 404 para id inexistente, segundo DELETE do mesmo id retorna 404).
- Melhoria adicional: `GET /version` devolve o nome do serviço e a versão (variável `APP_VERSION`, padrão 1.0.0); 1 teste. Total: 9 testes passando no CI.
- Fluxo de trabalho: branch `feature/delete-task-e-version` → PR #1 com CI aprovado → merge na `main`.

6. Evidências de funcionamento na nuvem

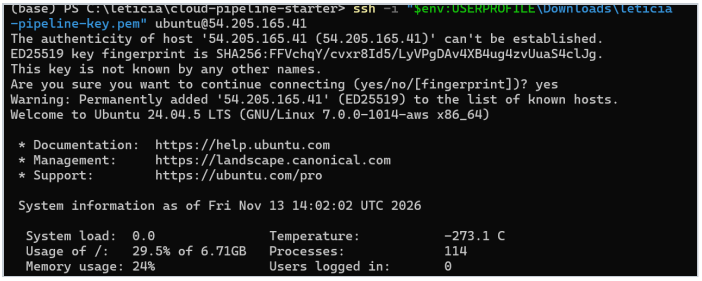


Fig. 8 — Acesso SSH à instância (Ubuntu 24.04.5 LTS).

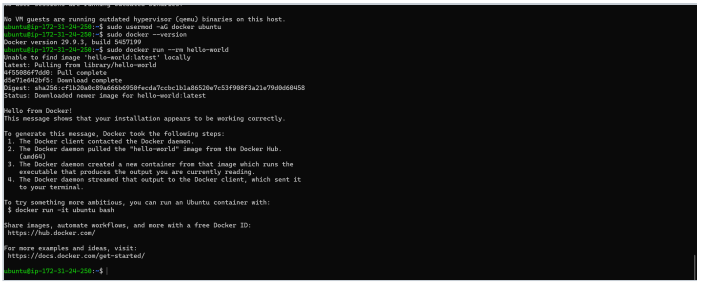


Fig. 9 — Docker 29.9.3 instalado e `hello-world` executado.

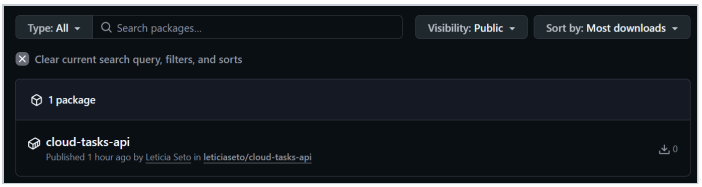


Fig. 10 — Pacote `cloud-tasks-api` no GHCR, visibilidade pública.

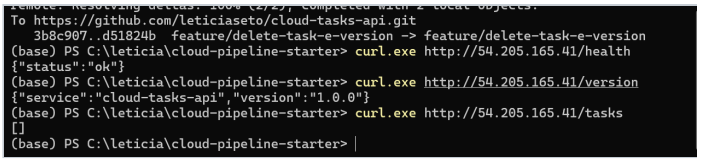


Fig. 11 — Acesso HTTP externo: `/health`, `/version` e `/tasks`.



Fig. 12 — `/version` acessado pelo navegador.

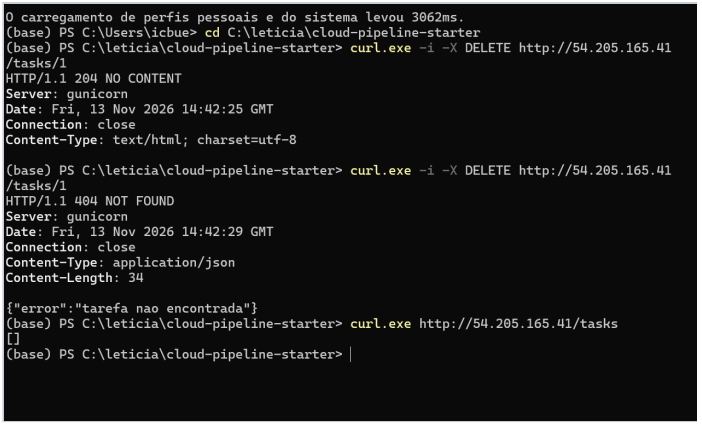


Fig. 13 — `DELETE`: 204 na primeira chamada e 404 na segunda; lista final vazia.

Rastreabilidade: o contêiner em execução usava a imagem `ghcr.io/leticiaseto/cloud-tasks-api:c97903da92ec…`, cujo SHA corresponde ao commit do merge (`c97903d`) exibido na Fig. 4.

Persistência: criei a tarefa *"Teste de persistencia"* (id 1), removi o contêiner com `docker rm -f`, confirmei que a lista de contêineres ficou vazia e que o volume `cloud-tasks-data` permaneceu, recriei o contêiner com o mesmo volume e a tarefa continuou disponível (Fig. 14).

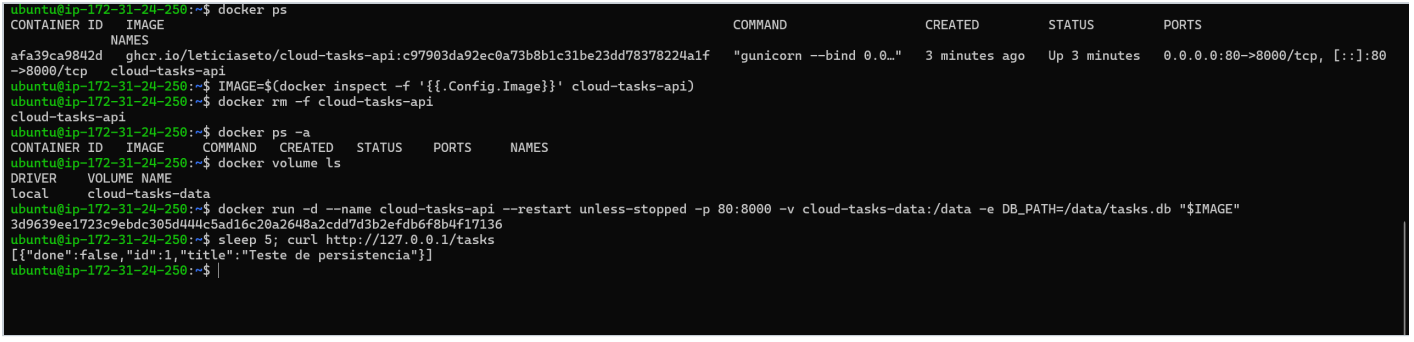


Fig. 14 — Prova de persistência: contêiner removido e recriado; a tarefa id 1 continua no banco.

7. Custos, recursos e encerramento

Custo. Foi usada uma única instância t3.micro, ligada das 10h38 às ~11h45 do dia 13/11/2026 (cerca de 1 h 10 min), com disco gp3 padrão da AMI. Estimando pelo preço de tabela sob demanda do t3.micro em us-east-1 (cerca de US$ 0,0104/h), o custo da instância fica em torno de US$ 0,01 a 0,02, mais valores desprezíveis de disco. Não foi usado IP elástico, balanceador, NAT nem banco gerenciado. O GHCR e o GitHub Actions foram usados dentro da cota gratuita de um repositório público. O saldo exibido no Learner Lab (Fig. 17) é acumulado de toda a conta e não representa só este projeto.

Encerramento, nesta ordem: (1) serviço do runner parado e desinstalado, runner removido do repositório (Fig. 15); (2) instância encerrada (Fig. 16); (3) grupo de segurança `cloud-pipeline-sg` excluído; (4) par de chaves `leticia-pipeline-key` excluído; (5) laboratório encerrado com *End Lab*. A VPC e as sub-redes padrão não foram alteradas.

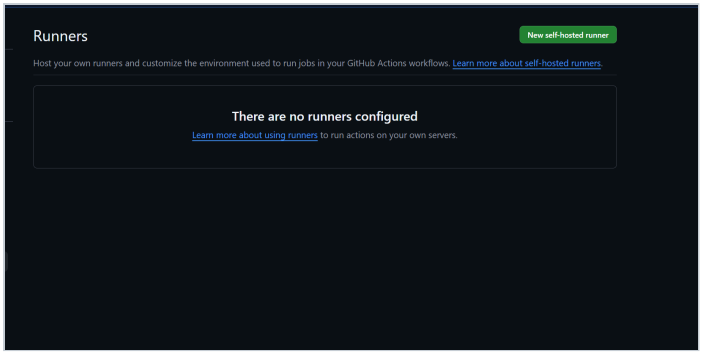


Fig. 15 — Nenhum runner configurado.

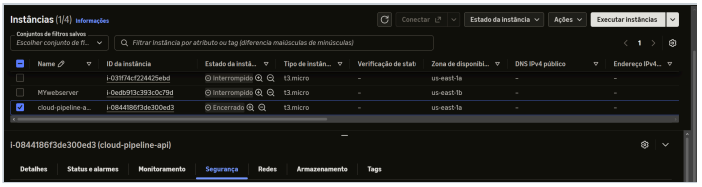


Fig. 16 — Instância `cloud-pipeline-api` em estado Encerrado.

Fig. 17 — Créditos do Learner Lab antes de encerrar o laboratório (saldo acumulado da conta).

8. Dificuldades encontradas

- **IP de origem instável e rede da faculdade.** O SSH e o HTTP deram *timeout* mesmo com as regras corretas: o IP público da rede da faculdade alternava entre dois endereços e, mesmo com os dois liberados, o acesso continuou falhando (provável filtro de saída da rede). Resolvi trocando para a rede móvel e liberando o IP observado nela também como `/32`, sem recorrer a `0.0.0.0/0`. Aprendizado: conferir o IP efetivo (`checkip`) e a rede antes de depurar a nuvem.
- **SSH a partir do GitHub.** Liberar a porta 22 para o GitHub contraria a restrição de segurança; a saída foi o runner self-hosted, com riscos documentados na seção 3.
- **Credencial Git em cache.** O primeiro `git push` falhou com 403 porque o Windows usava a credencial de outra conta; resolvi informando o usuário na URL do remoto.
- **Pacote GHCR.** O `docker pull` anônimo do script de deploy exige pacote público; verifiquei a visibilidade antes do merge (Fig. 10).
- **Segredos em capturas.** Capturas de tela podem expor IDs de conta, tokens e IPs; por isso foram recortadas ou mascaradas antes de entrarem neste relatório, e o token de registro do runner (uso único) já estava consumido.

9. Integrantes e divisão de tarefas

Integrante	Responsabilidades
Leticia Seto (@leticiaseto) — trabalho individual	Repositório e README; implementação de `DELETE /tasks/<id>` e `/version` com testes; workflows de CI e CD; provisionamento na AWS (EC2, grupo de segurança, Docker, runner); testes de persistência; encerramento dos recursos; evidências e relatório.

Demonstração em vídeo: não solicitada pelo docente; as evidências acima documentam o funcionamento e o encerramento.